

Math Nuggets for GEN AI

Supervised Fine-Tuning: Mathematical Foundations for LLM Adaptation

By Dr. Anish Roychowdhury, Plaksha University

September 25, 2025

From Pre-Training to Task-Specific Adaptation

Pre-trained large language models (LLMs) excel at general language tasks but often require adaptation to specific domains or tasks, such as question answering or summarization. Supervised Fine-Tuning (SFT) refines these models using curated datasets to optimize task-specific performance.

Consider the task: "Adapt a 7B parameter LLM for summarizing scientific articles."

Pre-training alone faces the following challenges for task-specific needs:

- **Generalization Gap:** Pre-trained models may lack domain-specific knowledge
- **Task Misalignment:** Outputs may not match desired format or style
- **Data Efficiency:** Large-scale pre-training data is not task-focused

Why SFT?

SFT bridges this gap by optimizing the model on a smaller, task-specific dataset, adjusting weights to improve performance while leveraging pre-trained knowledge.

SFT Architecture: Mathematical Framework

SFT optimizes an LLM's parameters using supervised learning on a dataset of prompt-response pairs.

Core Mathematical Formulation:

Let $\pi_\theta(y|x)$ represent the LLM, parameterized by θ , which predicts response y (a sequence of tokens) given prompt x . For a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, where each pair (x_i, y_i) is a prompt and its correct response, SFT minimizes the negative log-likelihood loss:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log \pi_{\theta}(y_{i,t} | x_i, y_{i,<t}) \quad (1)$$

Explaining Equation (1):

This equation calculates the loss that SFT tries to minimize. Think of it as a measure of how well the model predicts the correct tokens in the response. Here's what each part means:

- $\pi_{\theta}(y_{i,t} | x_i, y_{i,<t})$: The probability that the model assigns to the correct token $y_{i,t}$ (the t -th token in the i -th response) given the prompt x_i and all previous tokens $y_{i,<t}$. For example, if the response is "Sun rises," this is the probability of predicting "rises" after seeing "Sun."
- $\log \pi_{\theta}(\cdot)$: Taking the logarithm of the probability. Since probabilities are between 0 and 1, the log is negative, and we want to maximize this probability (make the log less negative).
- $-\sum_{t=1}^{T_i}$: Sum the negative log probabilities over all tokens in the response y_i , which has T_i tokens. The negative sign turns maximization into minimization.
- $\frac{1}{N} \sum_{i=1}^N$: Average the loss over all N prompt-response pairs in the dataset to get the overall loss.

In simple terms, this equation penalizes the model when it assigns low probabilities to correct tokens, encouraging it to learn better predictions.

The forward pass in a transformer-based LLM computes:

$$\mathbf{h}_t = \text{Transformer}(\mathbf{x}, \mathbf{y}_{<t}; \theta), \quad \pi_{\theta}(y_t | x, y_{<t}) = \text{Softmax}(\mathbf{W}_o \mathbf{h}_t) \quad (2)$$

Explaining Equation (2):

This equation describes how the LLM generates token probabilities during SFT:

- $\mathbf{h}_t = \text{Transformer}(\mathbf{x}, \mathbf{y}_{<t}; \theta)$: The transformer processes the input prompt $\mathbf{x}$ (encoded as a vector) and previous tokens $\mathbf{y}_{<t}$, using parameters θ , to produce a hidden state $\mathbf{h}_t$. This hidden state captures the context for predicting the next token.
- $\mathbf{W}_o$: A weight matrix that maps the hidden state to vocabulary scores. Each score represents how likely each word in the vocabulary is to be the next token.
- $\text{Softmax}(\mathbf{W}_o \mathbf{h}_t)$: Converts the scores into probabilities, ensuring they sum to 1. For example, if the vocabulary has four words, this might output probabilities like $[0.4, 0.3, 0.2, 0.1]$ for each word.

In essence, this equation shows how the model uses its transformer architecture to predict the next token and assign it a probability, which is then used in Equation (1) to compute the loss.

Optimization: Gradient descent (e.g., Adam) updates θ to minimize $\mathcal{L}(\theta)$, typically on a small dataset compared to pre-training.

SFT Pipeline Architecture

The SFT process is illustrated in Figure 1, showing the flow from pre-trained model to fine-tuned model.

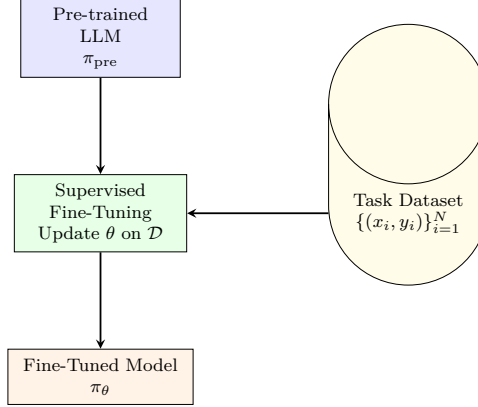


Figure 1: SFT pipeline showing dataset-driven optimization

Worked Example: SFT for Simple Summarization

Let's apply SFT to a toy example for summarizing a short text, focusing on predicting the first token to illustrate how fine-tuning improves the model's output.

Setup:

- Task: Summarize "The sun rises slowly behind the mountain, casting a warm glow." to "Sun rises behind mountain."
- Dataset: $\mathcal{D} = \{(x_1, y_1)\}$, where x_1 is the input text, y_1 is the target summary
- Model: Simplified linear layer with $\mathbf{W} \in \mathbb{R}^{4 \times 4}$, vocabulary size 4 (tokens: "Sun" [0], "rises" [1], "behind" [2], "mountain" [3])

Step 1: Input Representation

Encode x_1 as a vector $\mathbf{x} = [1 \ 0 \ 1 \ 0]^T$ (simplified token embeddings for the prompt).

Target y_1 tokens: ["Sun", "rises", "behind", "mountain"] mapped to indices [0, 1, 2, 3]. We focus on predicting the first token, "Sun" (index 0).

Step 2: Initial Weights

Pre-trained weight matrix:

$$\mathbf{W} = \begin{bmatrix} 1.0 & 0.3 & -0.2 & 0.5 \\ 0.2 & 0.9 & 0.4 & -0.1 \\ -0.5 & 0.1 & 1.2 & 0.3 \\ 0.7 & -0.2 & 0.1 & 1.1 \end{bmatrix} \quad (3)$$

Step 3: First Multiplication (Initial Prediction)

Compute the hidden state for the first token using the initial weights:

$$\mathbf{h}_1 = \mathbf{W}\mathbf{x} = \begin{bmatrix} 1.0 & 0.3 & -0.2 & 0.5 \\ 0.2 & 0.9 & 0.4 & -0.1 \\ -0.5 & 0.1 & 1.2 & 0.3 \\ 0.7 & -0.2 & 0.1 & 1.1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1.0 \cdot 1 + 0.3 \cdot 0 + (-0.2) \cdot 1 + 0.5 \cdot 0 \\ 0.2 \cdot 1 + 0.9 \cdot 0 + 0.4 \cdot 1 + (-0.1) \cdot 0 \\ -0.5 \cdot 1 + 0.1 \cdot 0 + 1.2 \cdot 1 + 0.3 \cdot 0 \\ 0.7 \cdot 1 + (-0.2) \cdot 0 + 0.1 \cdot 1 + 1.1 \cdot 0 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.6 \\ 0.7 \\ 0.8 \end{bmatrix} \quad (4)$$

Apply Softmax to get probabilities:

$$\pi_\theta(\text{token}_0) = \text{Softmax}(\mathbf{h}_1) = \text{Softmax} \left(\begin{bmatrix} 0.8 \\ 0.6 \\ 0.7 \\ 0.8 \end{bmatrix} \right) = \begin{bmatrix} \frac{e^{0.8}}{e^{0.8} + e^{0.6} + e^{0.7} + e^{0.8}} \\ \frac{e^{0.6}}{e^{0.8} + e^{0.6} + e^{0.7} + e^{0.8}} \\ \frac{e^{0.7}}{e^{0.8} + e^{0.6} + e^{0.7} + e^{0.8}} \\ \frac{e^{0.8}}{e^{0.8} + e^{0.6} + e^{0.7} + e^{0.8}} \end{bmatrix} \approx \begin{bmatrix} 0.27 \\ 0.22 \\ 0.24 \\ 0.27 \end{bmatrix} \quad (5)$$

The probability for "Sun" (index 0) is 0.27, tied for the highest with index 3 (0.27), but not strongly favoring the correct token. The loss for this token is $-\log(0.27) \approx 1.31$, indicating room for improvement.

Step 4: Update Weights

Gradient descent adjusts $\mathbf{W}$ to increase the probability of the correct token ("Sun"). After one update (simplified for illustration), the updated weight matrix is:

$$\mathbf{W}' \approx \begin{bmatrix} 1.05 & 0.31 & -0.19 & 0.51 \\ 0.21 & 0.92 & 0.41 & -0.09 \\ -0.48 & 0.11 & 1.22 & 0.31 \\ 0.71 & -0.19 & 0.11 & 1.12 \end{bmatrix} \quad (6)$$

The update slightly increases weights in the first row (corresponding to index 0, "Sun") for input dimensions where $\mathbf{x}$ has non-zero values (indices 1 and 3), emphasizing features relevant to predicting "Sun."

Step 5: Second Multiplication (Updated Prediction)

Compute the new hidden state with updated weights:

$$\begin{aligned} \mathbf{h}'_1 = \mathbf{W}'\mathbf{x} &= \begin{bmatrix} 1.05 & 0.31 & -0.19 & 0.51 \\ 0.21 & 0.92 & 0.41 & -0.09 \\ -0.48 & 0.11 & 1.22 & 0.31 \\ 0.71 & -0.19 & 0.11 & 1.12 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 1.05 \cdot 1 + 0.31 \cdot 0 + (-0.19) \cdot 1 + 0.51 \cdot 0 \\ 0.21 \cdot 1 + 0.92 \cdot 0 + 0.41 \cdot 1 + (-0.09) \cdot 0 \\ -0.48 \cdot 1 + 0.11 \cdot 0 + 1.22 \cdot 1 + 0.31 \cdot 0 \\ 0.71 \cdot 1 + (-0.19) \cdot 0 + 0.11 \cdot 1 + 1.12 \cdot 0 \end{bmatrix} = \begin{bmatrix} 0.86 \\ 0.62 \\ 0.74 \\ 0.82 \end{bmatrix} \quad (7) \end{aligned}$$

Apply Softmax to get updated probabilities:

$$\pi'_{\theta}(\text{token}_0) = \text{Softmax}(\mathbf{h}'_1) = \text{Softmax}\left(\begin{bmatrix} 0.86 \\ 0.62 \\ 0.74 \\ 0.82 \end{bmatrix}\right) \approx \begin{bmatrix} 0.29 \\ 0.23 \\ 0.26 \\ 0.28 \end{bmatrix} \quad (8)$$

Now, the probability for "Sun" (index 0) is 0.29, the highest in the vector, compared to 0.27 initially. This increase occurs because the updated weights $\mathbf{W}'$ amplify the hidden state value for index 0 (0.86 vs. 0.8), making the Softmax output favor "Sun" more strongly. The loss decreases to $-\log(0.29) \approx 1.24$, indicating better prediction.

Step 6: Efficiency Insight

SFT updates all parameters (16 here), but real LLMs with billions of parameters benefit from smaller datasets, reducing computational cost compared to pre-training.

Summary

SFT refines LLMs for specific tasks by optimizing on curated datasets:

- Adapts pre-trained models to domain-specific needs
- Leverages supervised learning for precise task alignment
- Uses small datasets for efficiency
- Serves as a foundation for advanced methods like RLHF

The mathematical simplicity of SFT lies in its log-likelihood optimization, enabling effective adaptation while retaining pre-trained capabilities. It's a cornerstone for LLM customization.

Practical Implementation Notes: Real-world SFT must address:

- **Dataset Quality** - ensuring high-quality, representative data
- **Overfitting** - balancing dataset size and model complexity
- **Format Consistency** - standardizing input-output pairs

References

- [1] DBLP:journals/corr/abs-2203-02155